T3000 API & Protocol

LAN Scanner — UDP Device Discovery · built Nov 13, 2026

LAN Scanner — UDP Device Discovery

Pure Rust async port of the C++ `RefreshNetWorkDeviceListByUDPFunc()` from `T3000-Source/T3000/global_function.cpp:9796`.

Protocol Overview

T3000 devices listen on **UDP port 1234** for a broadcast query and respond with device metadata. The scanner broadcasts `[0x64, 0,0,0,0]` to `255.255.255.255:1234` and collects `0x65` responses from all devices on reachable subnets.

Constant	Value	Meaning
`UPD_BROADCAST_QRY_MSG`	`0x64` (100)	Query command byte
`RESPONSE_MSG`	`0x65` (101)	Device response command
`RESPONSE_TOTAL_SUB_INFO`	`0x2F` (47)	Sub-device list response
`BROADCAST_DEST_PORT`	1234	Devices listen here
`SCAN_PORT_RANGE`	57619–57623	Scanner binds here for replies

Architecture

TEXT

```
api/src/lan_scan/
├── mod.rs          # Module entry, re-exports
├── types.rs        # DiscoveredDevice (20 fields), ScanResult, product_map (130+ entrie
├── protocol.rs     # Constants, build_scan_query(), parse_scan_response(), parse_sub_de
├── scanner.rs      # get_ip_mask_gateway(), scan_network(), scan_on_adapter(), bind_sca
└── tests/lan_scan/ # 18 unit tests + 1 real-network test (ignored)
```

Response Packet Layout

Each response is interleaved: data byte, reserve byte, data byte, … Offsets below are indices into the raw buffer.

Offset	Size	Field	Notes
0	1	command	`0x65`
4,6,8,10	4×u8	serial_number	Little-endian u32
12	1	product_id	Maps to product_name via product_map
14	1	modbus_id	
16,18,20,22	4×u8	ip_address	
24–25	2	modbus_port	Big-endian u16
26–27	2	firmware_version	/10 for products 10,35,50,62,74,88
28–29	2	hardware_version	
30–33	4	parent_serial	Airlab bug: equal bytes → 0
34,35,57,58	4	object_instance	
36	1	panel_number	
37–56	20	panel_name	Null-terminated ASCII
59	1	isp_mode	0=app, 1=bootloader→skip, 2=corrupted→skip
60–61	2	bacnet_ip_port	Big-endian u16
62	1	hardware_info	
63	1	subnet_protocol	0→1 mapping, 12→10 for BACnet devices
64	1	command_version	Only if buf.len ≥ 67
65	1	subnet_port	Only if parent≠0
66	1	subnet_baudrate	Only if parent≠0
67	1	minitype	Always at 67

Sub-Device Info (0x2F)

Offset	Size	Field
0	1	command = 0x2F
1	1	device_count
2–5	4	parent_serial (u32 LE)
6–20	15	reserved
21+	pairs	(status:u8, modbus_id:u8) × device_count

Scanning Flow

```text
scan_network(timeout_secs)
  ├─ get_local_ips() → enumerates local adapter IPs
  │    └─ Windows: ipconfig /all parsing
  │    └─ Fallback: connect UDP to 8.8.8.8
  └─ For each adapter IP:
      ├─ bind_scan_port(ip) → binds to ip:57619..57623
      ├─ set_broadcast(true)
      ├─ send_to(255.255.255.255:1234, query)
      └─ recv_loop(timeout):
          ├─ 0x65 → parse_scan_response() → DiscoveredDevice
          └─ 0x2F → parse_sub_device_info() → SubDeviceInfo
  └─ Deduplicate by serial_number
```

Key Differences from C++

Aspect	C++	Rust
Adapter enum	`GetAdaptersInfo()` Win32 API	`ipconfig /all` string parsing
Broadcast target	`255.255.255.255:1234`	Same ✅
Bind port	`57619-57623`	Same ✅
Socket model	Blocking `select()` + `recvfrom()`	Async Tokio `UdpSocket`
Timeout	Fixed 8-second `select()`	Configurable via parameter
Dedup	Tracks seen IPs in send buffer	`HashSet<u32>` after scan

Bugs Caught During Porting

1. **Broadcast port**: Sent to 57619 instead of 1234 — devices never heard queries
2. **IP parsing**: `(Preferred)` suffix from `ipconfig /all` broke `Ipv4Addr::parse()`
3. **Variable shadowing**: `current_ip = Some(v)` used old variable with suffix
4. **Subnet protocol mapping**: Was `6→0`, should be `0→1` (MODBUS_RS485→MODBUS_TCPIP)
5. **Minitype offset**: Was conditional on `parent_serial`, should always be `buf[67]`
6. **Sub-device base offset**: Was 22, should be 21
7. **Airlab parent_serial bug**: Equal bytes → zero (matches C++ quirk)
8. **Firmware /10**: Only for specific product IDs (10, 35, 50, 62, 74, 88)

Real Hardware Test Results

Verified against live T3000 devices on the LAN — 5 devices found across two subnets, matching the C++ desktop application output:

```text
192.168.0.x: 4 devices (SN 271292, 271851, 212375, 240488)
192.168.1.x: 1 device  (SN 237219)
```

Usage

```shell
## Unit tests
cargo test --test lan_scan

## Real network scan (requires T3000 hardware on LAN)
cargo test --test lan_scan test_real_network_scan -- --ignored --nocapture

## From code
use t3_webview_api::lan_scan::scanner;
let result = scanner::scan_network(8).await;  // 8 second timeout
```

C++ Reference Files

- `T3000-Source/T3000/global_function.cpp:9796` — `RefreshNetWorkDeviceListByUDPFunc()`
- `T3000-Source/T3000/global_function.cpp:9037` — `AddNetDeviceForRefreshList()` parser
- `T3000-Source/T3000/global_function.cpp:8982` — `AddSubNetInfoIntoRefreshList()` sub-device parser
- `T3000-Source/T3000/global_function.cpp:9610` — `GetIPMaskGetWay()` adapter enumeration

- `T3000-Source/T3000/global_define.h:619` — `refresh_net_device` struct
- `T3000-Source/T3000/global_define.h:244` — `UDP_BROADCAST_PORT = 1234`